1.

First, get a number from the user.

Check whether the number is greater than zero.

If the number is greater than zero, it is positive.

If the number is less than zero, it is negative.

Otherwise, the number is zero.

Result

2.

First, get a number from the user.

Initialize the sum as zero.

Separate the last digit from the number.

Add the last digit to the sum.

Remove the last digit from the number.

Repeat these steps until the number becomes zero.

Return the sum of the digits.

3.

First, get a number from the user.

Initialize the reverse value as zero.

Take the last digit of the number.

Add the digit to the reverse value by shifting the previous digits to the left.

Remove the last digit from the original number.

Repeat until the number becomes zero.

Return the reversed number.

4.

First, get a number from the user.

Check whether the number is less than or equal to one.

If it is less than or equal to one, it is not a prime number.

Otherwise, check whether the number is divisible by any number between 2 and one less than the given number.

If it is divisible by any of these numbers, it is not prime.

If it is not divisible by any of them, it is a prime number.

Return the result.

5.

First, get a number from the user.

Check whether the number is zero or one.

If the number is zero or one, return one.

Otherwise, multiply the number by the factorial of the previous number.

Repeat the process until the number reaches one.

Return the final factorial value.

6.

First, get a number from the user.

Count the total number of digits.

Separate each digit from the number.

Raise each digit to the power of the total number of digits.

Add all the calculated values.

Compare the sum with the original number.

If both are equal, it is an Armstrong number.

Otherwise, it is not an Armstrong number.

Return the result.

7.

First, get a string from the user.

Find the first character of the string.

Find the last character of the string.

Store the first character temporarily.

Replace the first character with the last character.

Replace the last character with the stored first character.

Keep the remaining characters unchanged.

Return the modified string.

8.

First, get a decimal number from the user.

Divide the number by two.

Find the remainder.

Store the remainder.

Divide the quotient by two again.

Repeat until the quotient becomes zero.

Arrange the stored remainders in reverse order.

Return the binary equivalent.

9.

First, get two strings from the user.

Convert both strings into lowercase.

Remove spaces if necessary.

Check whether both strings have the same number of characters.

If their lengths are different, they are not anagrams.

Otherwise, arrange the characters of both strings in alphabetical order.

Compare the two arranged strings.

If both are equal, they are anagrams.

Otherwise, they are not anagrams.

Return the result.